

Департамент образования и науки города Москвы
Государственное автономное образовательное учреждение
высшего образования города Москвы
«Московский городской педагогический университет»
Институт цифрового образования
Департамент информатики управления и технологий

Мошенина Елена Дмитриевна БД-241м

Инструменты хранения и анализа больших данных

Практическая работа 1.2. "Обработка данных с использованием Apache Spark и Python (PySpark)"

Направление подготовки/специальность
38.04.05 - Бизнес-информатика
Бизнес-аналитика и большие данные
(очная форма обучения)

Руководитель дисциплины:
Босенко Т.М., доцент департамента
информатики, управления и технологий,
доктор экономических наук

Москва
2025

Содержание

Введение.....	3
Основная часть.....	4
Заключение	27

Введение

Цель: освоение основ работы с Apache Spark и его интеграцией с Python через библиотеку PySpark. Студенты научатся обрабатывать большие объемы данных, используя распределенные вычисления, а также научатся применять базовые операции с RDD (Resilient Distributed Datasets) и DataFrame, работать с SQL-запросами в Spark SQL, а также визуализировать результаты обработки данных.

Задачи:

1. Установить Apache Spark и PySpark.
Настроить рабочую среду для использования PySpark в Python, установить необходимые зависимости и настроить Spark на локальном компьютере или через облачную платформу.
2. Загрузка данных и их предварительная обработка.
Скачать или подготовить исходные данные для анализа (например, текстовые файлы или CSV). Загружать данные в Spark через RDD или DataFrame, выполнить предварительную обработку: очистка данных, фильтрация, преобразования.
3. Применение операций с RDD и DataFrame.
Научиться работать с RDD и DataFrame, выполнять такие операции как map, filter, reduce, groupBy, join и другие стандартные операции для обработки данных в распределенной среде.
4. Применение SQL-запросов через Spark SQL.
Использование SQL-запросов в Spark для извлечения и агрегации данных, создание временных таблиц и выполнение сложных запросов для анализа данных.
5. Визуализация результатов анализа данных.
Визуализировать полученные результаты с помощью библиотеки Python для визуализации данных (например, matplotlib или seaborn). Построить графики для лучшего представления результатов.
6. Подготовка отчета.
Оформить отчет по выполненной практике, в котором будет описан процесс выполнения работы, анализ полученных результатов и выводы. Включить ссылки на репозиторий и прикрепить сам отчет в формате PDF или Markdown.

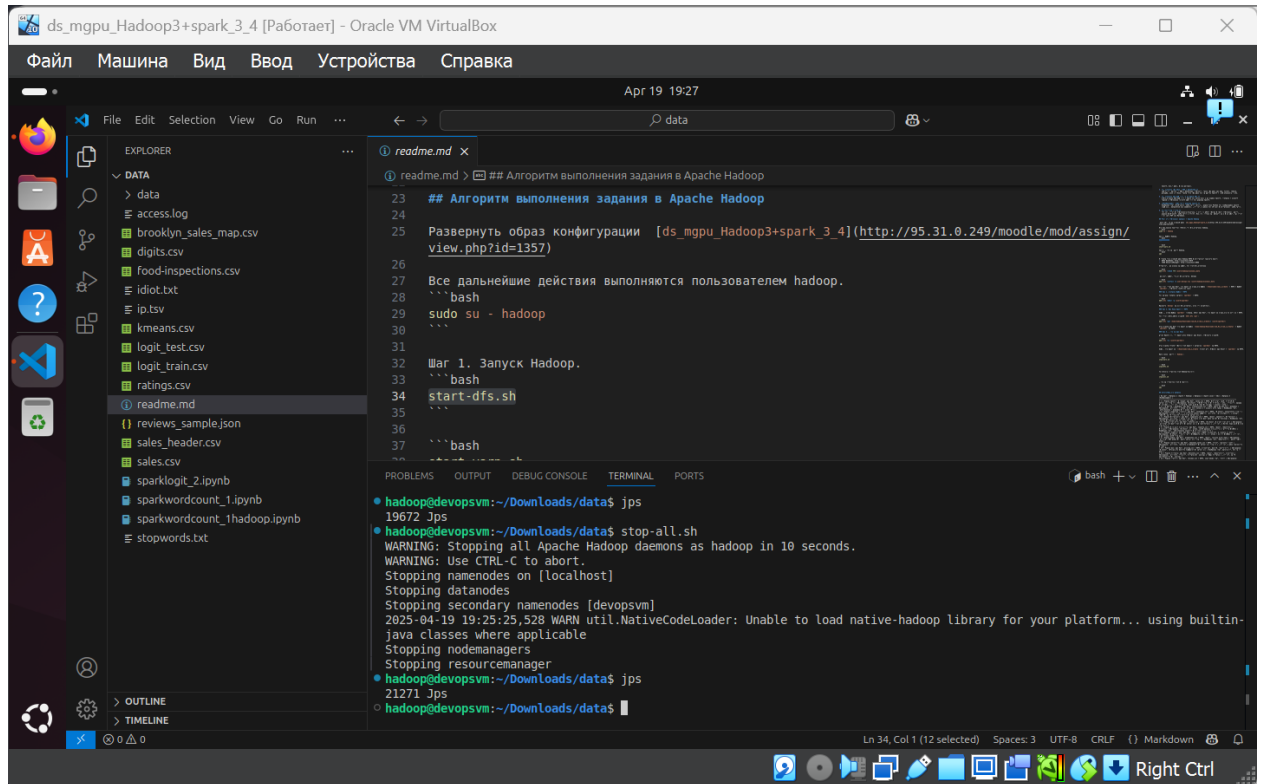
Основная часть

Алгоритм выполнения задания в Apache Hadoop

Развернуть образ конфигурации [ds_mgpu_Hadoop3+spark_3_4](http://95.31.0.249/moodle/mod/assign/view.php?id=1357)

Все дальнейшие действия выполняются пользователем `hadoop`.

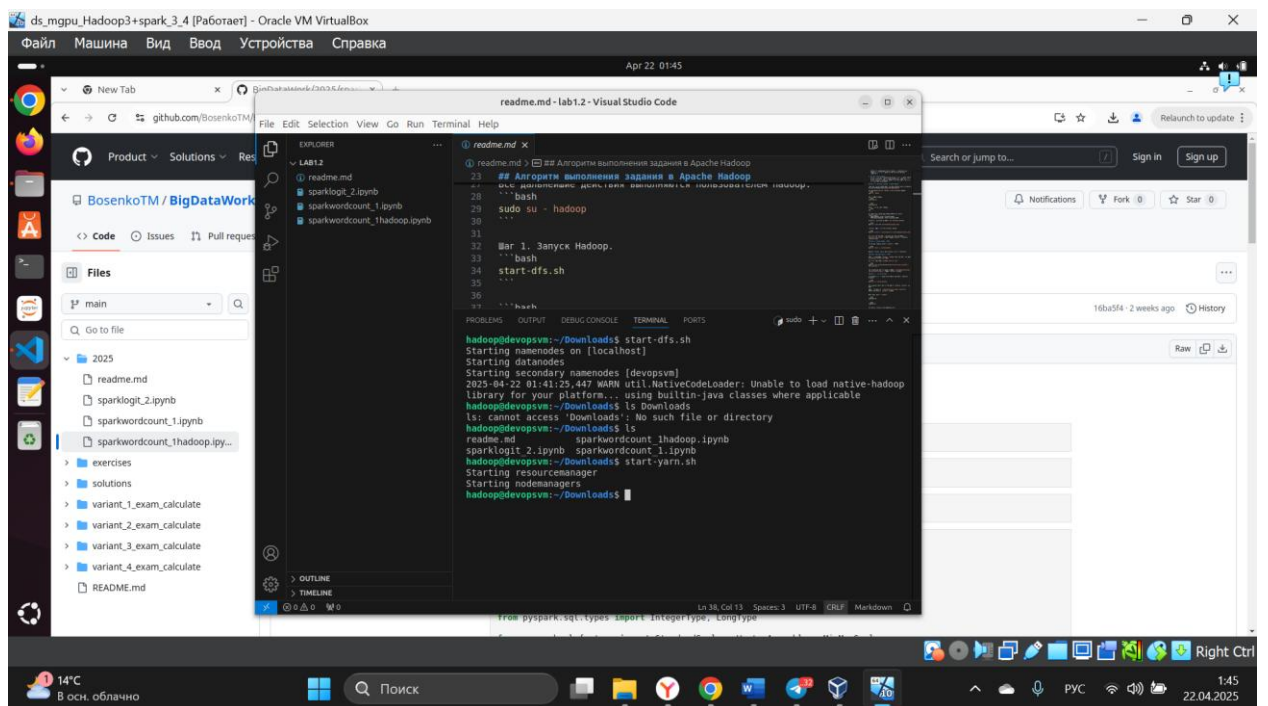
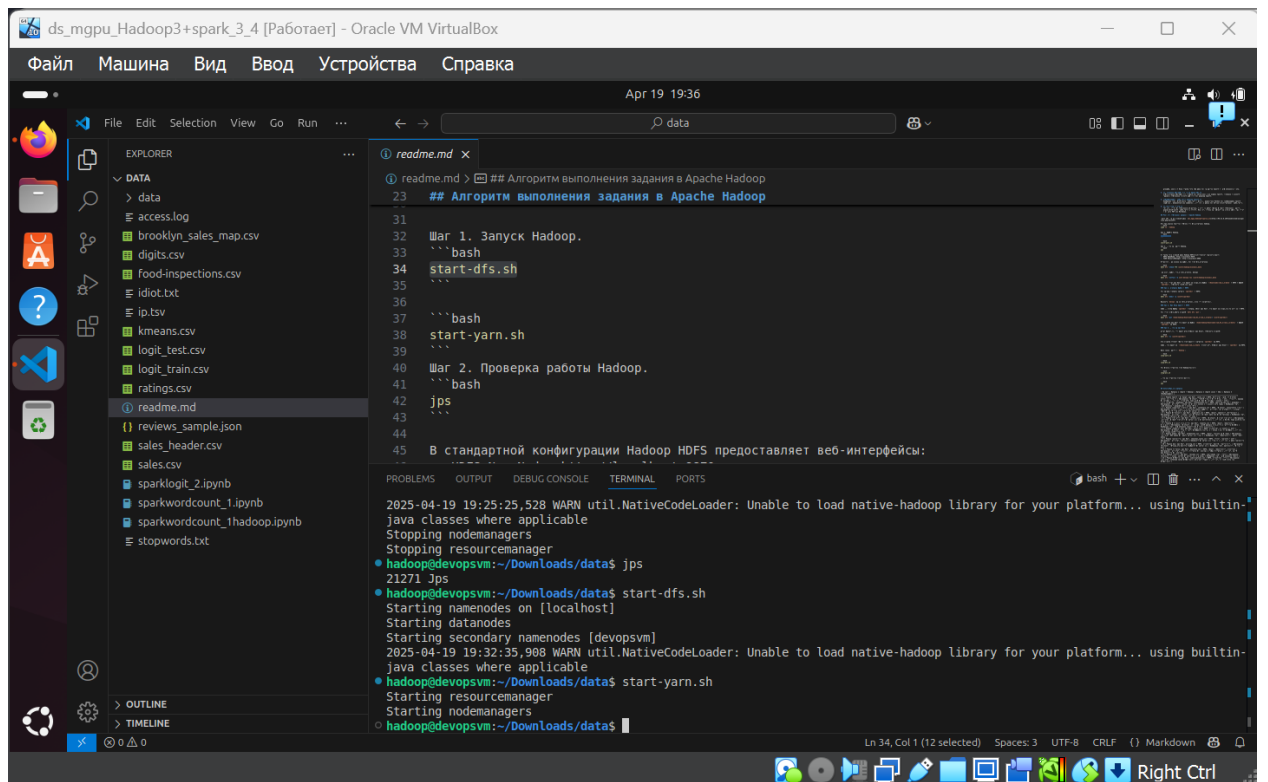
`sudo su – Hadoop`



Шаг 1. Запуск Hadoop.

`start-dfs.sh`

`start-yarn.sh`



Шаг 2. Проверка работы Hadoop.

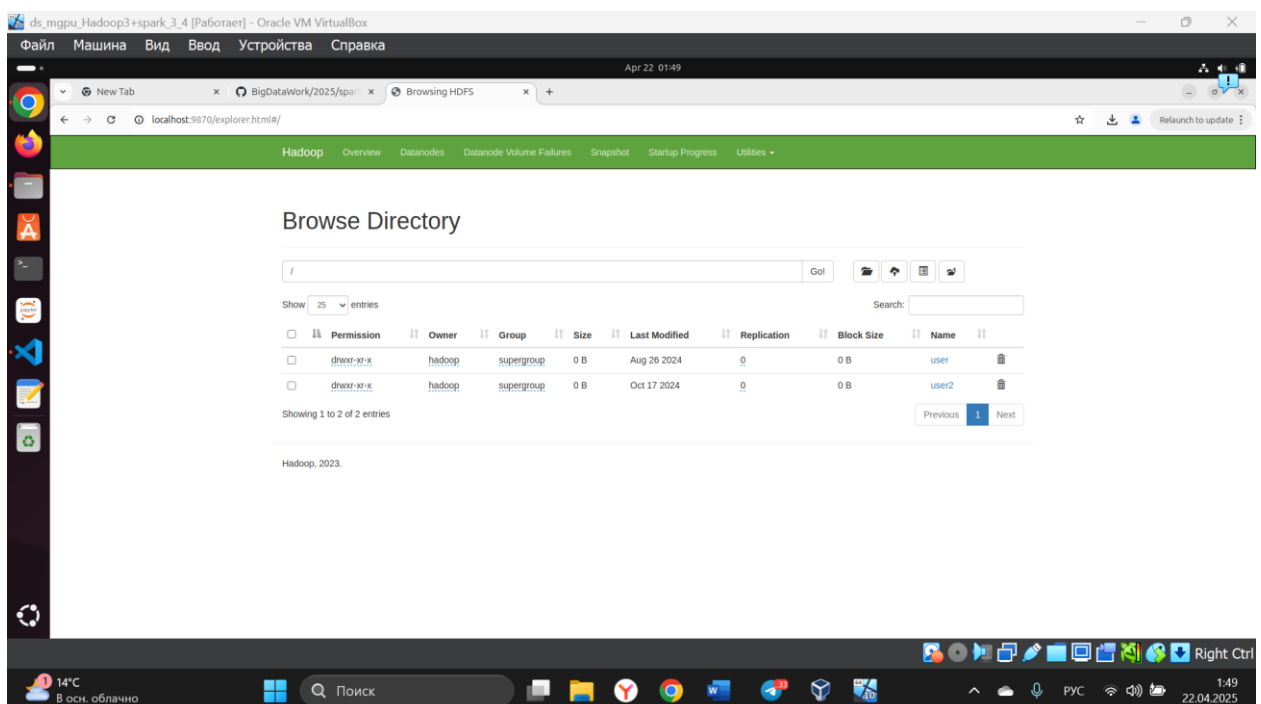
Jps

The screenshot shows the Visual Studio Code interface with a terminal window open. The terminal displays the following commands and output:

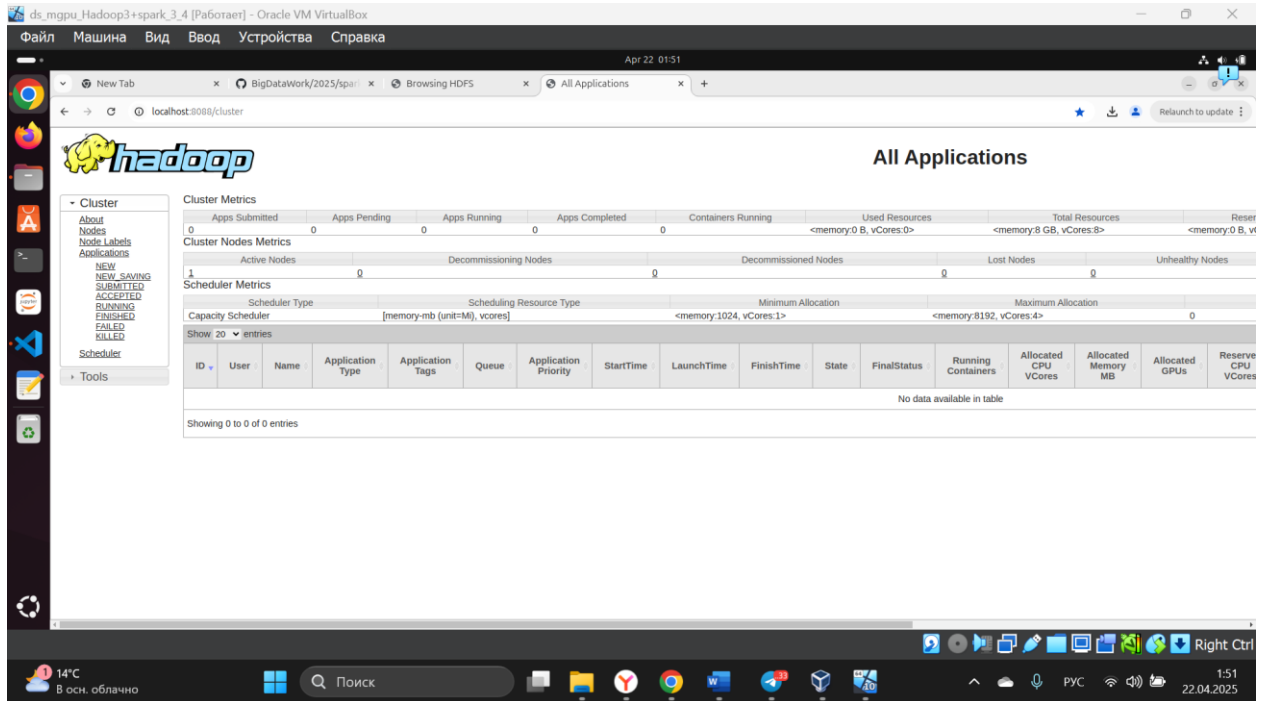
```
hadoop@devopsvm:~/Downloads$ start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [devopsvm]
2025-04-22 01:41:25,447 WARN util.NativeCodeLoader: Unable to load native-hadoop
library for your platform... using builtin-java classes where applicable
hadoop@devopsvm:~/Downloads$ ls Downloads
ls: cannot access 'Downloads': No such file or directory
hadoop@devopsvm:~/Downloads$ ls
readme.md      sparkwordcount_1hadoop.ipynb
sparklogit 2.ipynb  sparkwordcount_1.ipynb
hadoop@devopsvm:~/Downloads$ start-yarn.sh
Starting resourcemanager
Starting nodemanagers
hadoop@devopsvm:~/Downloads$ jps
13216 DataNode
14424 NodeManager
13512 SecondaryNameNode
13020 NameNode
14862 Jps
14254 ResourceManager
hadoop@devopsvm:~/Downloads$
```

В стандартной конфигурации Hadoop HDFS предоставляет веб-интерфейсы:

- HDFS NameNode: <http://localhost:9870>



- **YARN ResourceManager:** <http://localhost:8088>



установить разрешение на запись для всех пользователей

```
hdfs dfs -chmod 775 /user2/hadoop/economic_data
```

разрешить запись только пользователю devops

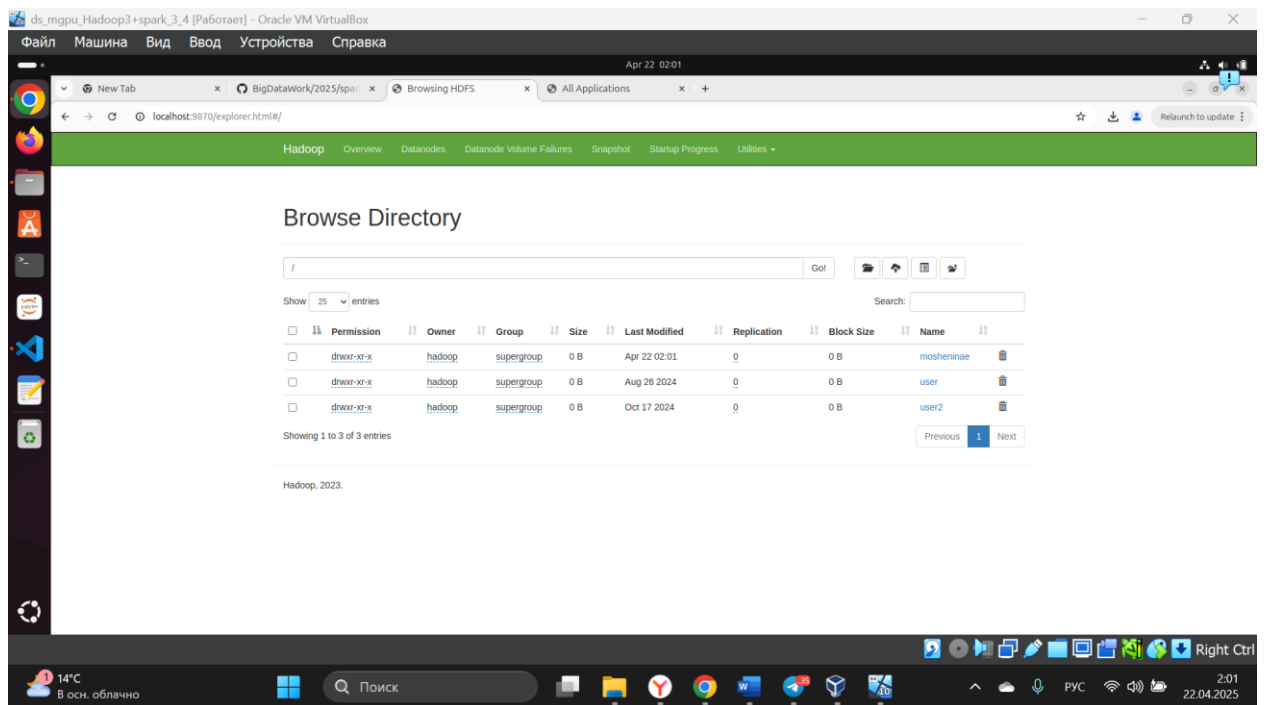
```
hdfs dfs -setfacl -m user:devops:rwX /user3/hadoop/economic_data
```

Для того чтобы загрузить все файлы из локальной папки ~/Downloads/lab_1_2/data в HDFS в папку sparkdir, выполните следующие шаги:

Шаг 1. Создание папки в HDFS

Для начала создадим каталог sparkdir в HDFS:

```
hdfs dfs -mkdir -p /user5/sparkdir
```



Замените devops на имя пользователя, если это необходимо.

Шаг 2. Загрузка файлов в HDFS

Теперь, когда папка sparkdir создана, нужно загрузить все файлы из локальной директории в HDFS.

Для этого используйте команду `hdfs dfs -put`:

```
hdfs dfs -put /home/hadoop/Downloads/lab_01_2/lab_1_2/data/* /user5/sparkdir/
```

readme.md - lab1.2 - Visual Studio Code

File Edit Selection View Go Run Terminal Help

EXPLORER

LAB1.2

- readme.md
- sparklogit_2.ipynb
- sparkwordcount_1.ipynb
- sparkwordcount_1hadoop.ipynb

readme.md

```
23 ## Алгоритм выполнения задания в Apache Hadoop
73 ### Шаг 2. Загрузка файлов в HDFS
80 hdfs dfs -put /home/hadoop/Downloads/lab_01_2/lab_1_2/data/* /
  user5/sparkdir/
81
82
83 Эта команда загрузит все файлы из папки `/home/hadoop/Downloads/
  lab_01_2/lab_1_2/data/` в папку `sparkdir` на HDFS.
84
85 ### Шаг 3. Проверка загрузки
86
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
-files <file1,...> specify a comma-separated list of files to be c
opied to the map reduce cluster
-libjars <jar1,...> specify a comma-separated list of jar files to
be included in the classpath
-archives <archive1,...> specify a comma-separated list of archives to b
e unarchived on the compute machines

The general command line syntax is:
command [genericOptions] [commandOptions]

Usage: hadoop fs [generic options] -mkdir [-p] <path> ...
hadoop@devopsvm:~/Downloads$ hdfs dfs -mkdir -p /mosheninae/test
2025-04-22 02:00:44,360 WARN util.NativeCodeLoader: Unable to load native-hadoop
library for your platform... using builtin-java classes where applicable
hadoop@devopsvm:~/Downloads$ hdfs dfs -put /home/hadoop/Downloads/data/* /mosheni
nae/test/
2025-04-22 02:06:55,418 WARN util.NativeCodeLoader: Unable to load native-hadoop
library for your platform... using builtin-java classes where applicable
put: `/home/hadoop/Downloads/data/*': No such file or directory
hadoop@devopsvm:~/Downloads$ hdfs dfs -put /home/hadoop/Downloads/* /mosheninae/t
est/
2025-04-22 02:07:55,412 WARN util.NativeCodeLoader: Unable to load native-hadoop
library for your platform... using builtin-java classes where applicable
hadoop@devopsvm:~/Downloads$
```

ds_mgpu_Hadoop3+spark_3.4 [Работает] - Oracle VM VirtualBox

Файл Машина Вид Ввод Устройства Справка

Apr 22 02:09

New Tab x BigDataWork/2025/spa x Browsing HDFS x All Applications x +

localhost:9870/explorer.html#/mosheninae/test

Hadoop Overview Datanodes Datanode Volume Failures Snapshot Startup Progress Utilities

Browse Directory

/mosheninaetest

Go

Show 25 entries

Search:

Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
-rw-r--r--	hadoop	supergroup	20.22 KB	Apr 22 02:08	1	128 MB	readme.md
-rw-r--r--	hadoop	supergroup	22.13 KB	Apr 22 02:08	1	128 MB	sparklogit_2.ipynb
-rw-r--r--	hadoop	supergroup	5.9 KB	Apr 22 02:08	1	128 MB	sparkwordcount_1.ipynb
-rw-r--r--	hadoop	supergroup	10.49 KB	Apr 22 02:08	1	128 MB	sparkwordcount_1hadoop.ipynb

Showing 1 to 4 of 4 entries

Previous 1 Next

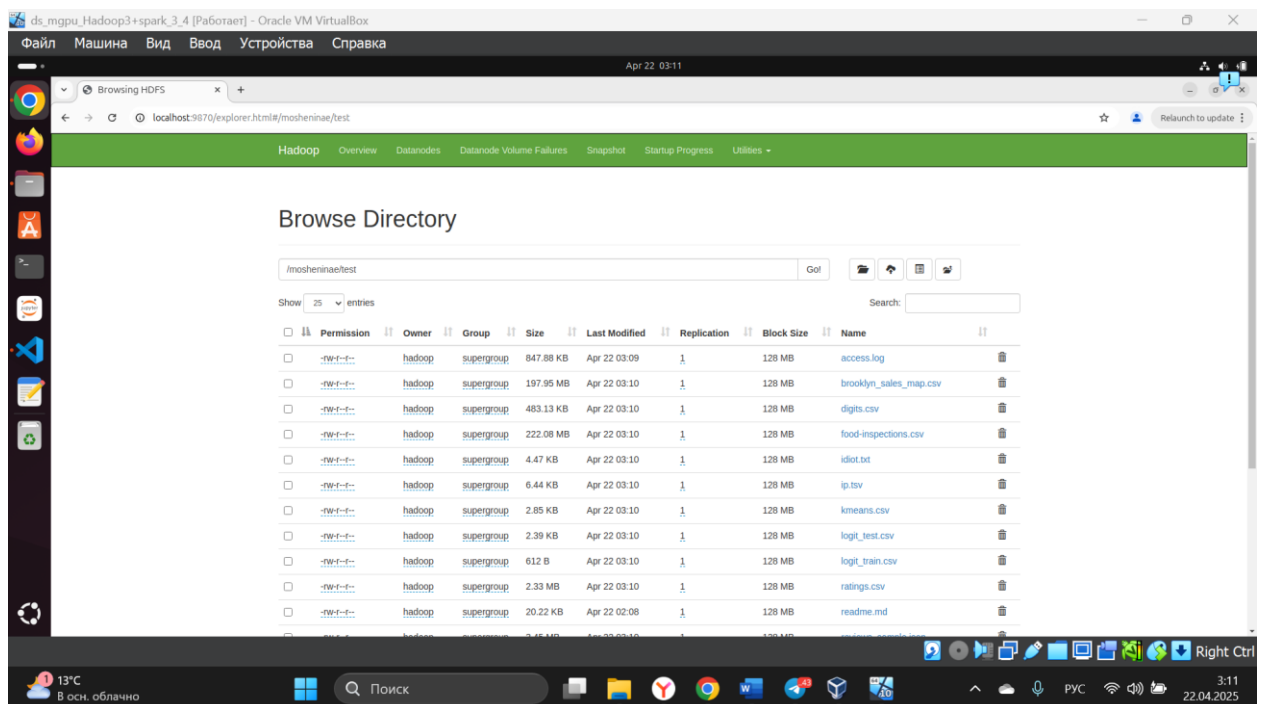
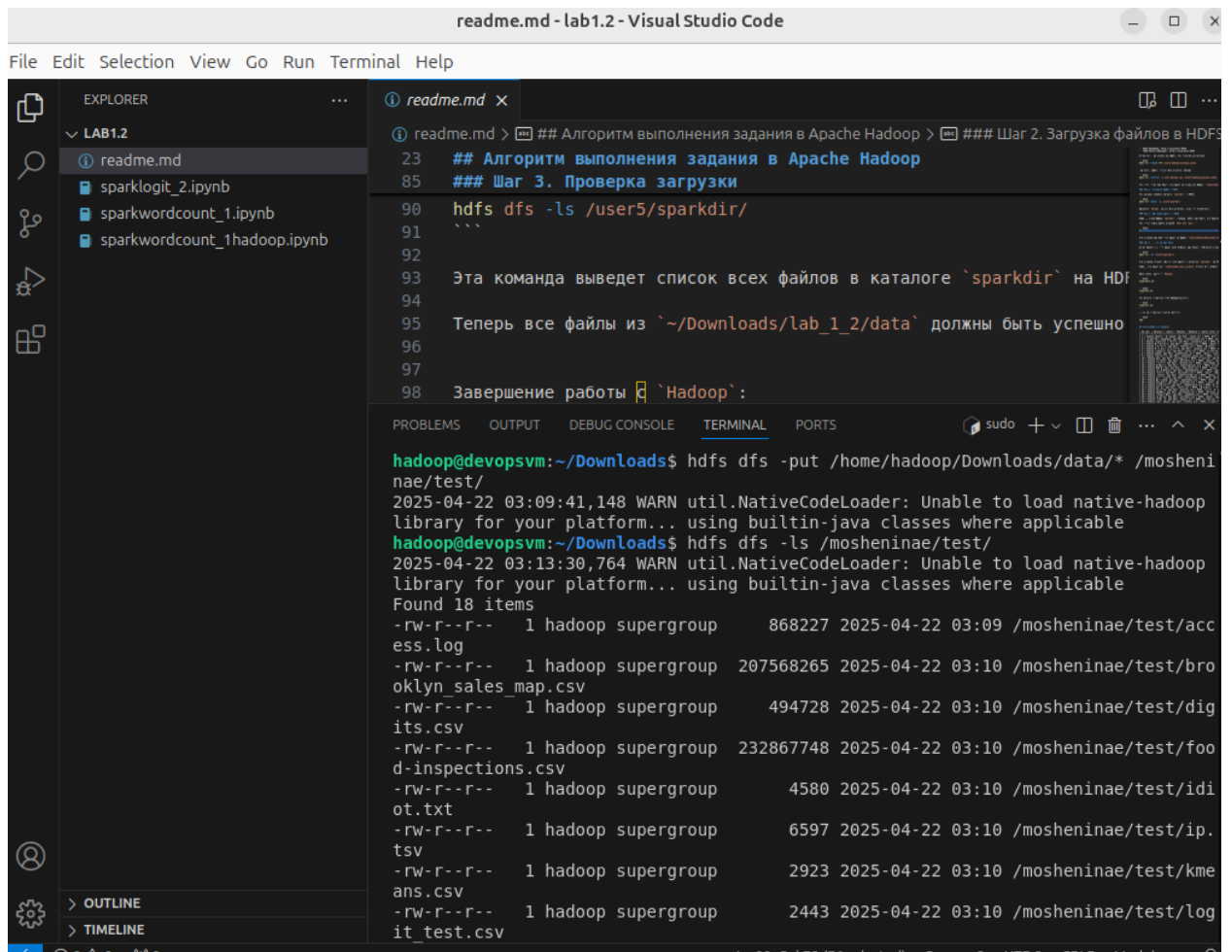
Hadoop, 2023.

14°C В осн. облачно

Поиск

2:09 22.04.2025

Эта команда загрузит все файлы из папки /home/hadoop/Downloads/lab_01_2/lab_1_2/data/ в папку sparkdir на HDFS.



Шаг 3. Проверка загрузки

Чтобы убедиться, что файлы были успешно загружены, выполните команду:

```
hdfs dfs -ls /user5/sparkdir/
```

```
readme.md - lab1.2 - Visual Studio Code
File Edit Selection View Go Run Terminal Help

EXPLORER
LAB1.2
  readme.md
  sparklogit_2.ipynb
  sparkwordcount_1.ipynb
  sparkwordcount_1hadoop.ipynb

readme.md
23  ## Алгоритм выполнения задания в Apache Hadoop
85  ### Шаг 3. Проверка загрузки
90  hdfs dfs -ls /user5/sparkdir/
91  ``
92
93  Эта команда выведет список всех файлов в каталоге `sparkdir` на HDFS.
94
95  Теперь все файлы из `~/Downloads/lab_1_2/data` должны быть успешно
96
97
98  Завершение работы с `Hadoop` :

TERMINAL
hadoop@devopsvm:~/Downloads$ hdfs dfs -put /home/hadoop/Downloads/data/* /mosheninae/test/
2025-04-22 03:09:41,148 WARN util.NativeCodeLoader: Unable to load native-hadoop
library for your platform... using builtin-java classes where applicable
hadoop@devopsvm:~/Downloads$ hdfs dfs -ls /mosheninae/test/
2025-04-22 03:13:30,764 WARN util.NativeCodeLoader: Unable to load native-hadoop
library for your platform... using builtin-java classes where applicable
Found 18 items
-rw-r--r-- 1 hadoop supergroup 868227 2025-04-22 03:09 /mosheninae/test/acc
ess.log
-rw-r--r-- 1 hadoop supergroup 207568265 2025-04-22 03:10 /mosheninae/test/bro
oklyn_sales_map.csv
-rw-r--r-- 1 hadoop supergroup 494728 2025-04-22 03:10 /mosheninae/test/dig
its.csv
-rw-r--r-- 1 hadoop supergroup 232867748 2025-04-22 03:10 /mosheninae/test/foo
d-inspections.csv
-rw-r--r-- 1 hadoop supergroup 4580 2025-04-22 03:10 /mosheninae/test/idi
ot.txt
-rw-r--r-- 1 hadoop supergroup 6597 2025-04-22 03:10 /mosheninae/test/ip.
tsv
-rw-r--r-- 1 hadoop supergroup 2923 2025-04-22 03:10 /mosheninae/test/kme
ans.csv
-rw-r--r-- 1 hadoop supergroup 2443 2025-04-22 03:10 /mosheninae/test/log
it test.csv
```

Эта команда выведет список всех файлов в каталоге sparkdir на HDFS.

Теперь все файлы из ~/Downloads/lab_1_2/data должны быть успешно загружены в sparkdir на HDFS.

Завершение работы с Hadoop:

stop-[yarn.sh](#)

stop-dfs.sh

Для полной остановки всех Hadoop-демонов:

stop-all.sh

Проверка остановки всех процессов:

jps

Индивидуальное задание

Анализ маркетплейса:
15 загрузить marketplace.csv в
HDFS, оценить эффективность
площадок

SQL-анализ: рассчитать комиссии и прибыль по площадкам

Построить сравнительный анализ площадок

Задание 1.

```
doc - JupyterLab
conda: jlab_server

File Edit View Run Kernel Tabs Settings Help

spark = SparkSession.builder \
    .appName("WordCount App") \
    .config("spark.hadoop.fs.defaultFS", "hdfs://localhost:9000") \
    .config("spark.ui.port", "4050") \
    .getOrCreate()

# Установка количества разделов для shuffle операций
spark.conf.set("spark.sql.shuffle.partitions", "50")

# Чтение данных из HDFS (текстовый файл)
file_path = "hdfs://localhost:9000/mosheninae/sparkdir/idiot.txt"
df = spark.read.text(file_path) # Используем .text, так как это текстовый файл

# Печать первых нескольких строк
df.show()
```

[]:

```
# Преобразование текста в RDD для подсчета частоты слов
text_rdd = df.rdd.flatMap(lambda row: row[0].split())

def clean_word(word):
    # Приводим слово к нижнему регистру
    word = word.lower()
    # Убираем все спецсимволы и цифры
    word = re.sub(r'[^a-zA-Za-zA-Zа-яА-ЯёЁ]', '', word) # Исправлено регулярное выражение
    # Убираем начальные и конечные пробелы
    word = word.strip()
    return word

# Очистка и фильтрация слов
text_rdd_cleaned = (text_rdd
                    .filter(lambda x: x is not None)
                    .map(lambda x: clean_word(x)) # Используем функцию очистки
                    .filter(lambda x: len(x) > 3) # Убираем слова длиной 3 и менее
                    .filter(lambda x: len(x) > 0) # Убираем пустые строки
```

```
doc - JupyterLab
conda: jlab_server

sparkwordcount_1hadoop.ipynb
File Edit View Run Kernel Tabs Settings Help

[1]: !pip install pyspark

Requirement already satisfied: pyspark in /home/devops/.config/jupyterlab-desktop/jlab_server/lib/python3.12/site-packages (3.5.3)
Requirement already satisfied: py4j==0.10.9.7 in /home/devops/.config/jupyterlab-desktop/jlab_server/lib/python3.12/site-packages (from pyspark) (0.10.9.7)

[2]: !pip install seaborn

Collecting seaborn
  Downloading seaborn-0.13.2-py3-none-any.whl.metadata (5.4 kB)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in /home/devops/.config/jupyterlab-desktop/jlab_server/lib/python3.12/site-packages (from seaborn) (1.26.4)
Requirement already satisfied: pandas>=1.2 in /home/devops/.config/jupyterlab-desktop/jlab_server/lib/python3.12/site-packages (from seaborn) (2.2.2)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in /home/devops/.config/jupyterlab-desktop/jlab_server/lib/python3.12/site-packages (from seaborn) (3.8.4)
Requirement already satisfied: contourpy>=1.0.1 in /home/devops/.config/jupyterlab-desktop/jlab_server/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.2.1)
Requirement already satisfied: cycler>=0.10 in /home/devops/.config/jupyterlab-desktop/jlab_server/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /home/devops/.config/jupyterlab-desktop/jlab_server/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.51.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /home/devops/.config/jupyterlab-desktop/jlab_server/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.5)
Requirement already satisfied: packaging>=20.0 in /home/devops/.config/jupyterlab-desktop/jlab_server/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (24.0)
Requirement already satisfied: pillow>=8 in /home/devops/.config/jupyterlab-desktop/jlab_server/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (10.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /home/devops/.config/jupyterlab-desktop/jlab_server/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.1.2)
Requirement already satisfied: python-dateutil>=2.7 in /home/devops/.config/jupyterlab-desktop/jlab_server/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.9.0)
```

```
doc - JupyterLab
conda: jlab_server

sparkwordcount_1hadoop.ipynb
File Edit View Run Kernel Tabs Settings Help

[4]: # Import Libraries
import pyspark
from pyspark import SparkConf
from pyspark.sql import SparkSession

from pyspark.sql.window import Window
from pyspark.sql.functions import udf, isnan, min, max, sum, count, desc, expr, avg
from pyspark.sql.types import IntegerType, LongType

from pyspark.ml.feature import StandardScaler, VectorAssembler, MinMaxScaler
from pyspark.ml.evaluation import MulticlassClassificationEvaluator
from pyspark.ml.tuning import CrossValidator, ParamGridBuilder, CrossValidatorModel
from pyspark.ml import Pipeline
from pyspark.ml.classification import DecisionTreeClassifier, RandomForestClassifier, LogisticRegression, GBTClassifier

import matplotlib.pyplot as plt
import seaborn as sns
import datetime
import numpy as np
import pandas as pd
import re # Добавляем импорт модуля re
from tabulate import tabulate
import matplotlib.pyplot as plt

[*]: from pyspark.sql import SparkSession

# Создание SparkSession
spark = SparkSession.builder \
    .appName("WordCount App") \
    .config("spark.hadoop.fs.defaultFS", "hdfs://localhost:9000") \
    .config("spark.ui.port", "4050") \
    .getOrCreate()

# Установка количества разделов для shuffle операций
spark.conf.set("spark.sql.shuffle.partitions", "50")
```

File Edit View Run Kernel Tabs Settings Help

 Python 3 (ipykernel)

```
[ ]: # Преобразование текста в RDD для подсчета частоты слов
text_rdd = df.rdd.flatMap(lambda row: row[0].split())

def clean_word(word):
    # Приводим слово к нижнему регистру
    word = word.lower()
    # Убираем все спецсимволы и цифры
```

doc - JupyterLab conda: jlab_server

File Edit View Run Kernel Tabs Settings Help

sparkwordcount_1hadoop.ipynb

Python 3 (ipykernel)

```
hue='word', # Явно указываем hue для устранения предупреждения
palette='viridis',
legend=False # Отключаем легенду
)
plt.title('Топ-10 наиболее часто встречающихся слов', pad=20)
plt.xlabel('Частота')
plt.ylabel('Слово')
plt.tight_layout()
plt.show()
```

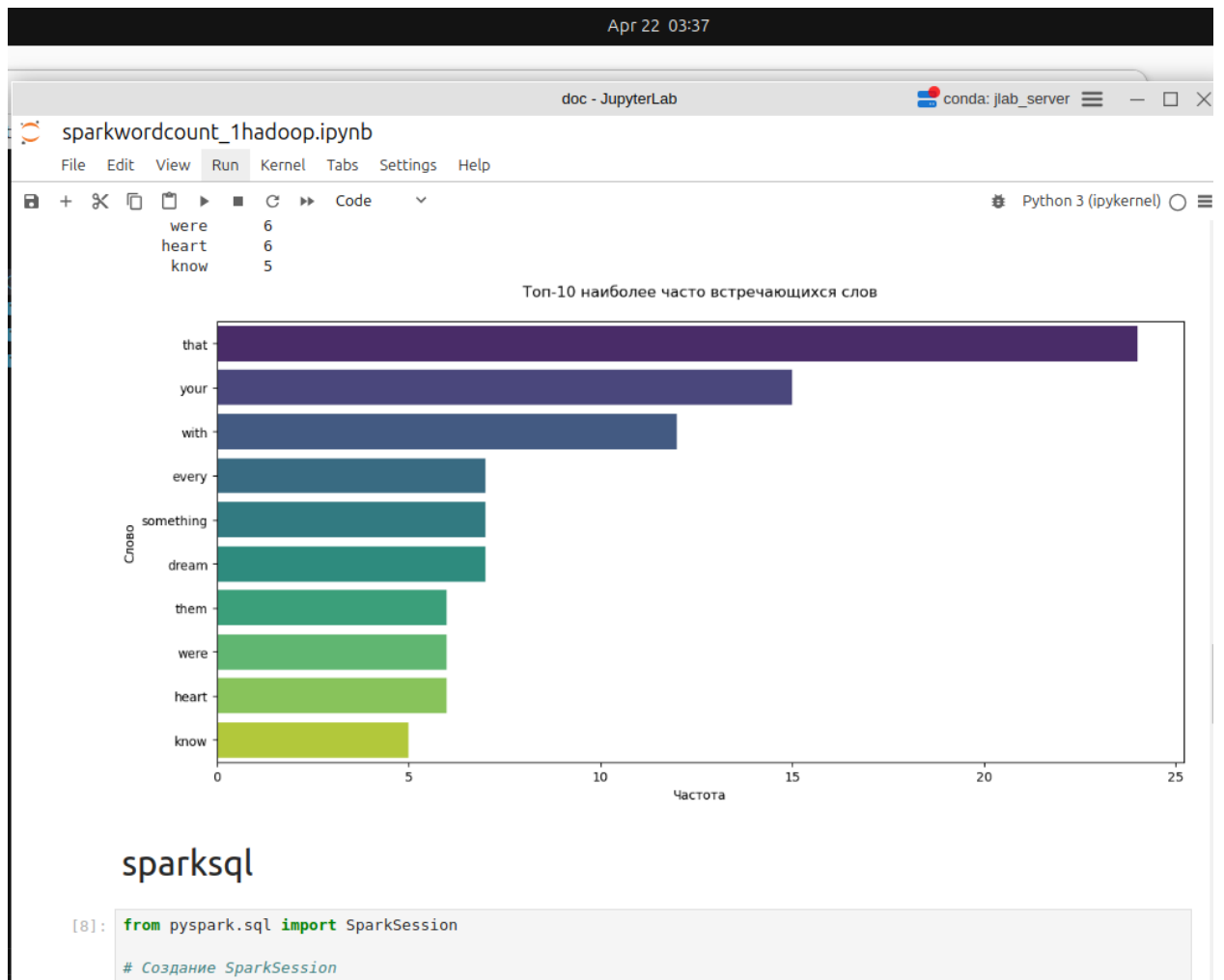
Топ-10 наиболее частых слов:

word	count
that	24
your	15
with	12
every	7
something	7
dream	7
them	6
were	6
heart	6
know	5

sparksqll

[]: from pyspark.sql import SparkSession

```
# Создание SparkSession
spark = SparkSession.builder \
    .appName("SQL App") \
    .config("spark.hadoop.fs.defaultFS", "hdfs://localhost:9000") \
    .config("spark.ui.port", "4050") \
    .getOrCreate()
```



Задание 2.

sparkwordcount_1hadoop.ipynb

File Edit View Run Kernel Tabs Settings Help

+ ✂ 📄 📄 ▶ ⏸ ↺ ⏹ Code ▾

Python 3 (ipykernel) ○ ≡

sparksql

[8]: `from pyspark.sql import SparkSession``# Создание SparkSession``spark = SparkSession.builder \`
 `.appName("SQL App") \`
 `.config("spark.hadoop.fs.defaultFS", "hdfs://localhost:9000") \`
 `.config("spark.ui.port", "4050") \`
 `.getOrCreate()``# Установка количества``spark.conf.set("spark``# Чтение данных из HDFS``file_path = "hdfs://l``header_path = "hdfs://`

Restart Kernel?

Do you want to restart the kernel of sparkwordcount_1hadoop.ipynb? All variables will be lost.

Cancel

Restart

25/04/22 03:37:29 WARN SparkSession: Using an existing Spark session; only runtime SQL configurations will take effect.

[9]: `def parse_row(line):``# Парсим строку (разделение по запятой)``return line.split(',')``# Чтение данных с использованием RDD``sales_rdd = spark.sparkContext.textFile(file_path).map(parse_row)``# Чтение заголовка из HDFS``sales_header_rdd = spark.sparkContext.textFile(header_path).take(1) # Читаем первую строку заголовка``sales_header = sales_header_rdd[0].split(',') # Разделяем по запятой`

sparksql

```
[8]: from pyspark.sql import SparkSession

# Создание SparkSession
spark = SparkSession.builder \
    .appName("SQL App") \
    .config("spark.hadoop.fs.defaultFS", "hdfs://localhost:9000") \
    .config("spark.ui.port", "4050") \
    .getOrCreate()

# Установка количества разделов для shuffle операций
spark.conf.set("spark.sql.shuffle.partitions", "50")

# Чтение данных из HDFS (текстовый файл)
file_path = "hdfs://localhost:9000/mosheninae/test/sales.csv"
header_path = "hdfs://localhost:9000/mosheninae/test/sales_header.csv"
```

25/04/22 03:37:29 WARN SparkSession: Using an existing Spark session; only runtime SQL configurations will take effect.

```
[9]: def parse_row(line):
    # Парсим строку (разделение по запятой)
    return line.split(',')

# Чтение данных с использованием RDD
sales_rdd = spark.sparkContext.textFile(file_path).map(parse_row)

# Чтение заголовка из HDFS
```

doc - JupyterLab

conda: jlab_server

sparkwordcount_1hadoop.ipynb

File Edit View Run Kernel Tabs Settings Help

Python 3 (ipykernel)

sparksql

```
[5]: from pyspark.sql import SparkSession

# Создание SparkSession
spark = SparkSession.builder \
    .appName("SQL App") \
    .config("spark.hadoop.fs.defaultFS", "hdfs://localhost:9000") \
    .config("spark.ui.port", "4050") \
    .getOrCreate()

# Установка количества разделов для shuffle операций
spark.conf.set("spark.sql.shuffle.partitions", "50")

# Чтение данных из HDFS (текстовый файл)
file_path = "hdfs://localhost:9000/mosheninae/test/sales.csv"
header_path = "hdfs://localhost:9000/mosheninae/test/sales_header.csv"
```

25/04/22 03:44:25 WARN Utils: Your hostname, devopsvm resolves to a loopback address: 127.0.1.1; using 10.0.2.15 instead (on interface enp0s3)
25/04/22 03:44:25 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
25/04/22 03:44:35 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable

```
[6]: def parse_row(line):
    # Парсим строку (разделение по запятой)
    return line.split(',')
```

sparkwordcount_1hadoop.ipynb

File Edit View Run Kernel Tabs Settings Help

+ ✖ 📄 📄 ▶ ■ ↺ ⏪ Code ▾

Python 3 (ipykernel) ○ ≡

```
# Выполнение SQL-запроса для получения всех данных
sales_total_df = spark.sql("SELECT * FROM sales")

# Печать первых 10 строк
sales_total_df.show(10)
```

```
+-----+-----+-----+-----+-----+-----+-----+
|Transaction_date| Product|Price|Payment_Type|      Name|      City|      State|      Country|
|Account_Created| Last_Login| Latitude| Longitude|-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+
| 1/2/09 6:17|Product1| 1200| Mastercard| carolina| Basildon| England|United Kingdom|
1/2/09 6:00| 1/2/09 6:08|      51.5|-1.116667|
| 1/2/09 4:53|Product1| 1200| Visa| Betina|Parkville| ...| MO| United States|
1/2/09 4:42| 1/2/09 7:49|      39.195| -94.68194|
| 1/2/09 13:08|Product1| 1200| Mastercard|Federica e Andrea|Astoria| ...| OR| United States|
1/1/09 16:21|1/3/09 12:32| 46.18806| -123.83|
| 1/3/09 14:44|Product1| 1200| Visa| Gouya| Echuca| Victoria| Australia|
9/25/05 21:13|1/3/09 14:22|-36.1333333| 144.75|
| 1/4/09 12:56|Product2| 3600| Visa| Gerd W |Cahaba Heights| ...| AL| United States|
11/15/08 15:47|1/4/09 12:45| 33.52056| -86.8025|
| 1/4/09 13:19|Product1| 1200| Visa| LAURENCE|Mickleton| ...| NJ| United States|
9/24/08 15:19|1/4/09 13:04|      39.79| -75.23806|
| 1/4/09 20:11|Product1| 1200| Mastercard| Fleur|Peoria| ...| IL| United States|
1/3/09 9:38|1/4/09 19:45| 40.69361| -89.58889|
| 1/2/09 20:09|Product1| 1200| Mastercard| adam|Martin| ...| TN| United States|
1/2/09 17:43|1/4/09 20:01| 36.34333| -88.85028|
| 1/4/09 13:17|Product1| 1200| Mastercard| Renee Elisabeth| Tel Aviv| Tel Aviv| Israel|
1/4/09 13:03|1/4/09 22:10| 32.0666667|34.7666667|
| 1/4/09 14:11|Product1| 1200| Visa| Aidan| Chatou|Ile-de-France| France|
6/3/08 4:22| 1/5/09 1:17| 48.8833333| 2.15|
+-----+-----+-----+-----+-----+-----+
only showing top 10 rows
```

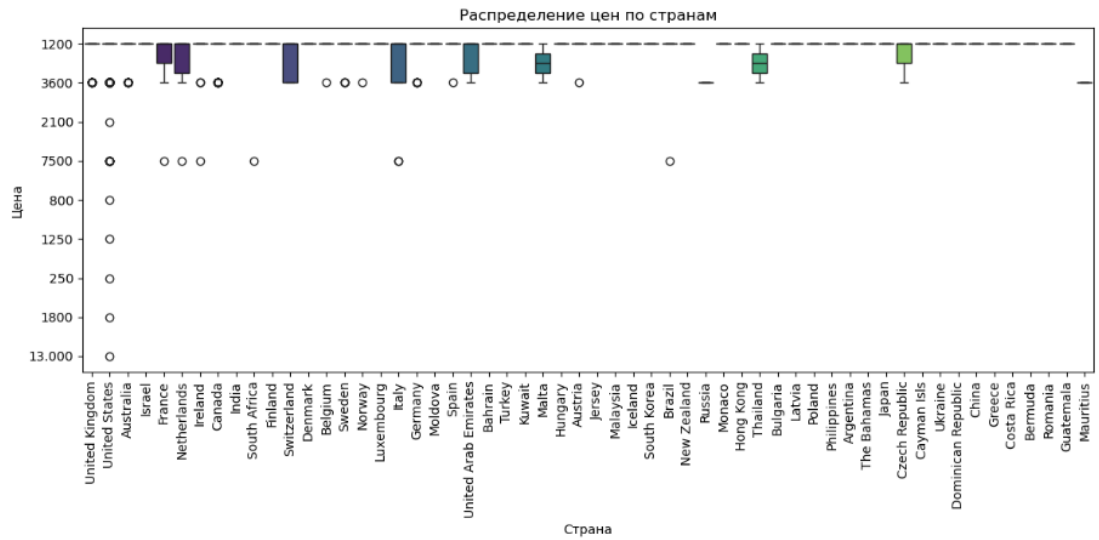
sparkwordcount_1hadoop.ipynb

File Edit View Run Kernel Tabs Settings Help

Code

Python 3 (ipykernel)

sns.boxplot()



```
[ ]: # TODO: Вывод дохода по стране и штату
revenue_by_country_state = spark.sql("""
    SELECT Country, State, SUM(Price) as Total_Revenue
    FROM sales
    GROUP BY Country, State
    ORDER BY Total_Revenue DESC
""")
```

Apr 22 03:49

doc - JupyterLab

conda: jlab_server

sparkwordcount_1hadoop.ipynb

File Edit View Run Kernel Tabs Settings Help

Python 3 (ipykernel)

```
ORDER BY Total_Revenue DESC
"""
# Отображаем результаты
revenue_by_country_state.show()
```

Country	State	Total_Revenue
United Kingdom	England	120000.0
United States	CA	113350.0
United States	NY	61200.0
United States	TX	55500.0
United States	FL	51600.0
Canada	Ontario	46800.0
United States	VA	40400.0
Canada	British Columbia	28800.0
Ireland	Dublin	28800.0
United States	GA	28200.0
Canada	Alberta	26400.0
United States	WA	24000.0
United States	IL	24000.0
Netherlands	Zuid-Holland	23100.0
United States	NJ	22800.0
United States	MD	22800.0
Australia	New South Wales	20400.0
United States	PA	20400.0
United States	TN	19500.0
United States	MN	19200.0

only showing top 20 rows

Задание 2.

colab.research.google.com

sparklogit_2.ipynb - Colab

sparklogit_2.ipynb

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
[6] import os

os.chdir("/content/drive/My Drive/data")
os.listdir()
```

```
['digits.csv',
 'kmeans.csv',
 'ip.tsv',
 'stopwords.txt',
 'access.log',
 'idiot.txt',
 'sales_header.csv',
 'logit_train.csv',
 'logit_test.csv',
 'sales.csv']
```

Теперь необходимо импортировать входные данные, создав на их основе набор RDD (Resilient Distributed Dataset).

0 сек. выполнено в 04:03

4:04 22.04.2025

10 диссер

colab.research.google.com

sparklogit_2.ipynb - Colab

Поделиться

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

ОЗУ Диск

Теперь необходимо импортировать входные данные, создав на их основе набор RDD (Resilient Distributed Dataset).

34 сек.

[8] import findspark
findspark.init()
from pyspark.sql import SparkSession
spark = SparkSession.builder.master("local[*]").getOrCreate()
data = spark.read.csv(
 '/content/drive/My Drive/data/brooklyn_sales_map.csv',
 inferSchema=True, header=True)

Готовим атрибут для последующей бинарной классификации

Допустим, требуется классифицировать налоговый класс на дом (tax_class). Всего имеется 10 таких классов. Поскольку данные распределены неравномерно (например, в классе 1 имеется 198969 записей, а в 3-м — только 18), мы разделим их на 2 категории: те, которые принадлежат классу 1, и остальные. В Python это делается очень просто, нужно просто вызвать метод replace:

+ Код + Текст

[9] by_1 = ['1', '1A', '1B', '1C']
by_others = ['2', '2A', '2B', '2C', '3', '4']
data = data.replace(by_others, '0', ['tax_class'])

9 сек. выполнено в 07:32

10 диссер

colab.research.google.com

sparklogit_2.ipynb - Colab

Поделиться

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

ОЗУ Диск

0 сек.

[9] by_1 = ['1', '1A', '1B', '1C']
by_others = ['2', '2A', '2B', '2C', '3', '4']
data = data.replace(by_others, '0', ['tax_class'])
data = data.replace(by_1, '1', ['tax_class'])

Кроме того, алгоритмы Machine Learning в PySpark работают с числовыми значениями, а не со строками. Поэтому преобразуем значения столбца tax_class в тип int:

0 сек.

[10] data = data.withColumn('tax_class', data.tax_class.cast('int'))

Подбор признаков и преобразование категорий

Выберем следующие признаки для обучения модели Machine Learning: год постройки (year_of_sale), цена на дом (sale_price) и соседние регионы (neighborhood). Последний атрибут является категориальным признаком — в данных имеется 29 соседних регионов. Но опять же все значения этих категорий являются строковыми, поэтому нужно преобразовать их в числовые.

Можно воспользоваться методом replace, как это сделано выше, но придётся сначала извлечь названия всех 29 регионов. А можно использовать специальный класс StringIndexer из PySpark-модуля ml, который выполнит за нас всю работу. Объект этого

9 сек. выполнено в 07:32

10 диссер sparklogit_2.ipynb - Colab

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

ОЗУ Диск

```
data.groupBy('neighborhood_id').count().show()
```

neighborhood_id	count
8.0	13215
0.0	27279
7.0	13387
49.0	2271
29.0	5074
47.0	2422
42.0	3086
44.0	2802
35.0	4000
18.0	7342
1.0	21206
39.0	3396
37.0	3894
34.0	4037
25.0	5809
36.0	3984
41.0	3138
4.0	14608
23.0	6374
56.0	985

9 сек. выполнено в 07:32

Поиск

10 диссер sparklogit_2.ipynb - Colab

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

ОЗУ Диск

Теперь выберем необходимые признаки, а также отбросим строки с пустыми значениями с помощью метода dropna в PySpark:

```
[13] features = ['year_of_sale', 'sale_price', 'neighborhood_id']
      target = 'tax_class'
      attributes = features + [target]
      sample = data.select(attributes).dropna()
```

ВЕКТОРИЗАЦИЯ ПРИЗНАКОВ

Поскольку алгоритмы машинного обучения в PySpark принимают на вход только вектора, то нужно провести векторизацию. Для преобразования признаков в вектора используется класс `VectorAssembler`. Объект этого класса принимает в качестве аргументов список с названиями признаков, которые нужно векторизовать (`inputCols`), и название преобразованного признака (`outputCol`). После создания объекта `VectorAssembler` вызывается метод `transform`.

Для начала выберем в качестве признака для преобразования — цену на дом. Код на Python:

```
[14] from pyspark.ml.feature import VectorAssembler
```

9 сек. выполнено в 07:32

Поиск

10 диссер colab.research.google.com sparklogit_2.ipynb - Colab

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

ОЗУ Диск

Для начала выберем в качестве признака для преобразования цену по дом. код на 1 уикон.

```
[14] from pyspark.ml.feature import VectorAssembler

assembler = VectorAssembler(inputCols=['sale_price'],
                             outputCol='features')
output = assembler.transform(sample)
```

Полученный после векторизации DataFrame выглядит следующим образом:

```
output.show(5)
```

year_of_sale	sale_price	neighborhood_id	tax_class	features
2008	4.99401179E8	48.0	0	[4.99401179E8]
2016	3.45E8	41.0	0	[3.45E8]
2016	3.4E8	27.0	0	[3.4E8]
2012	2.76947E8	52.0	0	[2.76947E8]
2017	2.025E8	27.0	0	[2.025E8]

only showing top 5 rows

9 сек. выполнено в 07:32

7:36 22.04.2025

10 диссер colab.research.google.com sparklogit_2.ipynb - Colab

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

ОЗУ Диск

```
[17] from pyspark.ml.classification import LogisticRegression

lr = LogisticRegression(featuresCol='features',
                         labelCol='tax_class')
model = lr.fit(train)
```

Осталось только получить предсказания. Для этого вызывается метод transform, который принимает тестовую выборку:

```
[18] predictions = model.transform(test)
```

Проверим эффективность модели, используя метрику качества. И в этом случае PySpark нас выручает, поскольку у него есть класс BinaryClassificationEvaluator. Нужно лишь указать целевой признак (tax class), а затем вызвать метод evaluate и передать в него наши предсказания. В Python это выглядит так:

```
[19] from pyspark.ml.evaluation import BinaryClassificationEvaluator

evaluator = BinaryClassificationEvaluator(labelCol='tax_class')
print('Evaluation:', evaluator.evaluate(predictions))
```

Evaluation: 0.5250699659768593

9 сек. выполнено в 07:32

7:36 22.04.2025

Векторизуем также год постройки (year_of_sale) и соседние регионы (neighborhood_id). Для этого нужно только в VectorAssembler указать выбранные признаки:

```
[20] features = ['year_of_sale', 'sale_price', 'neighborhood_id']

assembler = VectorAssembler(inputCols=features,
                             outputCol='features')
output = assembler.transform(sample)
```

```
output.show(5)
```

year_of_sale	sale_price	neighborhood_id	tax_class	features
2008	4.99401179E8	48.0	0	[2008.0, 4.9940117...
2016	3.45E8	41.0	0	[2016.0, 3.45E8, 41.0]
2016	3.4E8	27.0	0	[2016.0, 3.4E8, 27.0]
2012	2.76947E8	52.0	0	[2012.0, 2.76947E...
2017	2.025E8	27.0	0	[2017.0, 2.025E8, 2...

only showing top 5 rows

9 сек. выполнено в 07:32

```
[22] train, test = output.randomSplit([0.8, 0.2])

[23] from pyspark.ml.classification import LogisticRegression

lr = LogisticRegression(featuresCol='features',
                         labelCol='tax_class')
model = lr.fit(train)
```

```
predictions = model.transform(test)
```

```
from pyspark.ml.evaluation import BinaryClassificationEvaluator

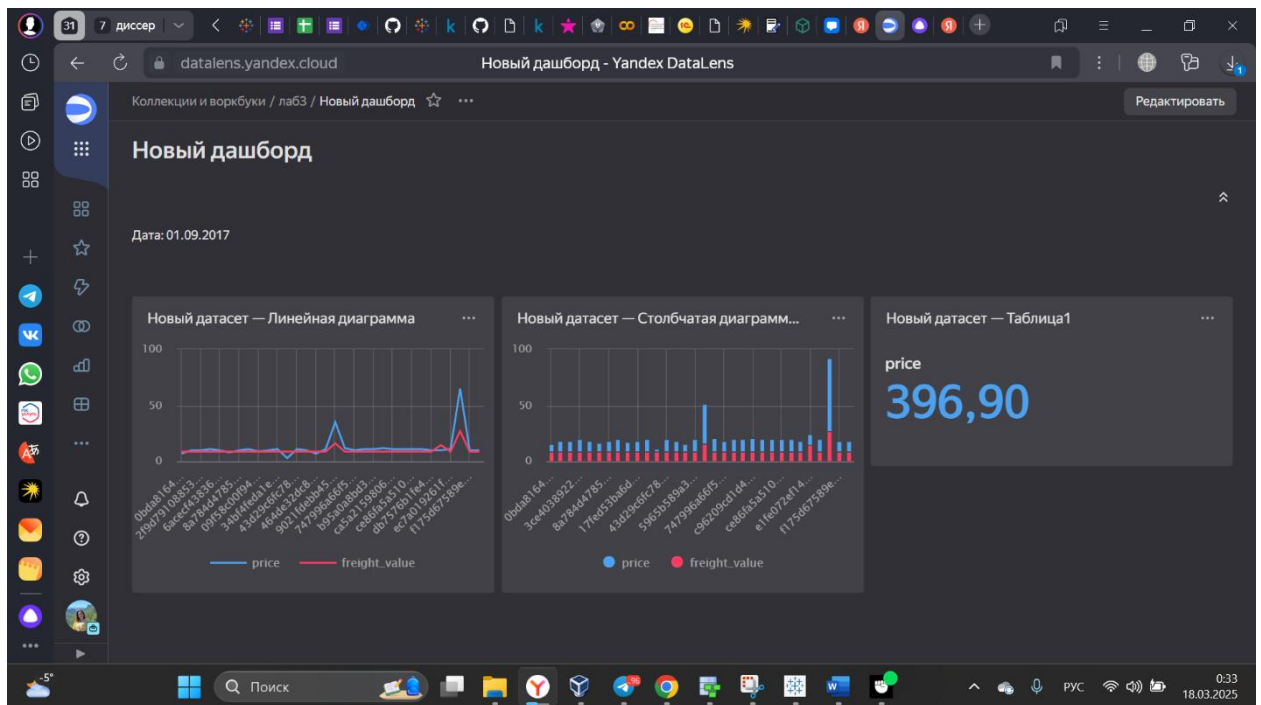
evaluator = BinaryClassificationEvaluator(labelCol='tax_class')
print('Evaluation:', evaluator.evaluate(predictions))
```

Evaluation: 0.6061912236859169

Python-код для остальных шагов – разделение на тестовую и обучающую выборки, обучение и оценивание модели – остаётся все тем же. В итоге, мы смогли повысить точность до 60%:

9 сек. выполнено в 07:32

Задание 3.



Заключение

В результате освоения материала успешно освоили основы работы с Apache Spark и научились интегрировать его с Python посредством библиотеки PySpark. В ходе обучения были достигнуты следующие результаты:

- Получили практические навыки работы с распределенными вычислениями и научились эффективно обрабатывать большие объемы данных
- Освоены базовые операции с **Resilient Distributed Datasets (RDD)** и **DataFrame**, что позволяет работать с данными в различных форматах
- Приобретены навыки работы с **Spark SQL**, включая выполнение SQL-запросов для анализа данных
- Изучены методы визуализации результатов обработки данных, что позволяет наглядно представлять полученные результаты

В процессе обучения научились:

- Создавать и управлять RDD
- Выполнять преобразования и действия над датасетами
- Работать с DataFrame для структурированной обработки данных
- Применять SQL-запросы для анализа данных в Spark

- Настраивать и оптимизировать работу Spark-приложений
- Визуализировать результаты обработки для дальнейшего анализа

Полученные знания и навыки позволят эффективно решать задачи по обработке больших данных, используя мощь распределенных вычислений Spark в сочетании с гибкостью Python. Это открывает широкие возможности для дальнейшей работы в области анализа данных и машинного обучения.

Таким образом, поставленные цели обучения полностью достигнуты, и мы готовы применять полученные знания на практике при работе с большими данными.